Transaction Coordinator (TC) AI Tool

Complete Build Requirements — Everything Needed to Build It End to End

Version 1.0 · Compiled 2026-07-12

---

0. How to read this document

This is the exhaustive inventory of **every single thing** required to build the TC tool — from empty machine to a production MVP, with the path to the full product mapped at the end. It consolidates the build roadmap, the spec extracts, and the gaps that neither document covers.

Three build targets are referenced throughout:

- **Target A — Synthetic MVP:** the §11 vertical slice working end to end on synthetic data through real screens. This is the near-term goal.
- **Target B — Production launch:** the same slice handling *real* client NPI, which adds legal, security, and operational requirements.
- **Target C — Full product:** everything deferred in §7 of the roadmap (portals, multi-state, integrations). Out of MVP scope by design.

Status tags used in this document:

- `[DONE]` — completed in the current build.
- `[WIP]` — started, incomplete.
- `[TODO]` — not started; buildable with what you have.
- `[OBTAIN]` — information that exists in the spec but you have not surfaced yet.
- `[EXTERNAL]` — cannot be produced by writing code or docs; requires a lawyer, domain expert, or a vendor's approval.
- `[BLOCKER]` — gates a specific later phase or launch.

---

1. Product definition and scope

What it is (SOR positioning). The TC tool is a **system of record (SOR)** for California residential real-estate transactions that replaces the transaction coordinator's mental switchboard. It ingests a signed purchase agreement from a scoped, dedicated deal address, extracts the deal's fields, builds a deadline timeline from human-verified California rules, and drafts outbound follow-ups that a human approves before sending. Every party, document, deadline, task, message, and approval lives in one append-only, audited record — the deal's single source of truth.

MVP trigger. Email-monitoring via a scoped, dedicated deal address (`deal-xxxx@...`) that TCs forward or BCC to, parsed via Postmark inbound. Manual upload is retained as a fallback for unreadable emails or scans. Personal-mailbox OAuth scanning is explicitly **not** in the MVP.

Core design commitments. Three decoupled parts talking only through payloads; human-in-the-loop (HITL) on transaction creation and updates; synthetic-data-only testing; five hard rules enforced by an automated auditor.

---

2. The five hard rules (verbatim — never violate)

1. **Only ever ingest the customer's own signed contract. Never generate, host, or pre-fill blank C.A.R. (California Association of Realtors) forms.** Why: reading a signed copy is permitted; reproducing blank copyrighted forms is not.
2. **Never touch money movement or wiring instructions.** Why: wire fraud is the top loss vector in real estate closings; automating it is unacceptable risk.
3. **A human approves every outbound message. No auto-send, ever.** Why: a wrong message to an outside party is the thing practitioners most fear.
4. **California residential only to start.** Why: each state's forms and deadline rules differ; we get one right first.
5. **Any AI model that sees the documents runs in a no-train / no-retain configuration.** Why: the files contain Social Security numbers and bank details; that data must never be retained or trained on.

---

3. External accounts, services, and credentials (Stage A)

Every account required, what it is for, the credentials it produces, and where each credential must live. **Rule: real keys go only in a gitignored `.env` or a password manager — never in code, never committed, never in an AI's context.**

Service	Purpose	Status
Supabase	Postgres (SOR), auth + MFA, encrypted document storage, row-level security	`[DONE]`
Anthropic API	Field extraction + message drafting	`[DONE]` (key live)
Postmark	Inbound parse (MVP trigger) + outbound send	`[DONE]`
GitHub	Private repo + version control	`[DONE]`
Domain registrar	Branded deal address + email deliverability	`[TODO]` (deferred; using Postmark defaults)
Backend host	Run FastAPI in production	`[TODO]`

Frontend host	Serve the React app	[TODO]
Error/monitoring (e.g. Sentry)	Production observability	[TODO]

Full credential inventory (secrets — store safely, never commit):

- `SUPABASE_URL`, `SUPABASE_ANON_KEY` (frontend-safe only because RLS is on), `SUPABASE_SERVICE_ROLE_KEY` (backend only — bypasses RLS), Supabase DB password.
- `ANTHROPIC_API_KEY` (backend only).
- `POSTMARK_SERVER_TOKEN` (backend only), plus the inbound deal address and the verified outbound "from" address (not secret, but track them).
- GitHub Personal Access Token (fine-grained, repo-scoped) — local git auth.
- Supabase personal access token — for the Supabase MCP (read-only, project-scoped).

Hard gate (safety). Until Anthropic zero-data-retention is confirmed active — and forever, for how data is handled — no real client email, document, SSN, or bank detail enters the API, test fixtures, logs, or an AI's context. Build and test on synthetic data only.

---

4. Local development environment and prerequisites

- macOS (Apple Silicon). [DONE]
- **Node.js 18+** — required for the React frontend and Node-based MCP servers. (v24 installed.) [DONE]
- **Python 3.10+** — FastAPI backend. (3.13 present.) [DONE]
- **git** configured with user name/email. [DONE]
- **Claude Code CLI** installed and run from inside the project repo. [DONE]
- Per-service virtual environments (backend `.venv`) and `node_modules` — gitignored.
- A password manager for all secrets. [DONE] (assumed)

---

5. Claude Code workspace (Stage B)

- **`CLAUDE.md`** — SOR positioning, five rules verbatim, three-part architecture, stack, TDD mandate, "propose a new rule" line. [DONE]
- **`rules/`** — `security.md`, `architecture.md`, `testing.md`, `data-model.md`, `code-style.md`. [DONE]
- **Four subagents** (`.claude/agents/`): `ca-rules-researcher`, `test-writer`, `code-reviewer`, `compliance-auditor`. [DONE]
- **MCPs:** Supabase MCP (read-only, project-scoped) [DONE]; Chrome DevTools MCP for UI verification [TODO] (turn on for Phases 2 and 7); optional GitHub MCP and Playwright MCP [TODO/optional].
- **Optional skills:** `new-component`, `migration` — add only if repeated.

6. System architecture

Three decoupled parts. They communicate only through validated Payloads. No part reads or writes another part's database, and no part imports another part's code.

- **(a) Ingestion agent** — watches the dedicated deal address (Postmark inbound) and the manual-upload fallback; detects doc type / readability / signatures; extracts fields with confidence; emits a Payload. Never writes the SOR directly.
- **(b) Master / SOR** — the Supabase-backed authoritative record. Consumes Payloads after HITL confirmation.
- **(c) Compliance / timeline service** — interval-run; computes Deadlines/Tasks from human-verified CA rules; raises Risk Flags. Never guesses the rules.

The Payload contract (interface between a and b): which document, which transaction (or "new"), which party, and extracted fields `[{name, value, confidence, confirmed?}]`. Transaction creation from a "new" payload is HITL.

Stack. React + TypeScript (frontend); Python / FastAPI (backend); Supabase (Postgres, auth+MFA, encrypted storage, RLS); Anthropic API (Claude, no-train/no-retain); Postmark (inbound parse + outbound send).

7. Data model (§10) — the single source of truth

Entities:

- **Transaction** — one home sale; has one Property, many Parties, Documents, Deadlines, Tasks, Messages, Payloads, and an Audit Log. Identified by a transaction ID.
- **Property** — the home (address, included/excluded items, possession).
- **Party / Contact** — a person or company on the deal, with a role that sets its permission tier. Includes multiple inspection parties (general, termite/pest, roof, sewer) and an optional Contractor for post-inspection repairs.
- **Document** — an uploaded/ingested file, access-controlled by side/role; may carry a pending state until confirmed.
- **Payload** — validated package handed from ingestion to master (document, transaction, party, extracted fields, confidence).
- **Extracted Field** — one value read from a document, with a confidence score and a `confirmed?` flag.
- **Deadline** — a dated obligation computed from confirmed fields.
- **Task** — a thing to do, tied to a Deadline, with status and dependencies; can belong to a receiving-end party.
- **Message** — an email drafted/sent to a Party, with `draft / approved / sent` state.
- **Reminder** — a scheduled nudge tied to a Deadline or Task.

- **Risk Flag** — a warning raised by the compliance service against a Task/Deadline.
- **Approval** — the record that a human approved a Message (or pending inbound item) before it committed.
- **Audit Log** — append-only history of who did what and when.

Relationships. A Transaction has a Property and many Parties; ingested Documents arrive as Payloads that become Extracted Fields (scored, confirmed); confirmed fields create Deadlines then Tasks, Reminders, and Risk Flags; Tasks generate Messages that require an Approval to send; everything writes to the Audit Log.

---

8. Permission tiers (§8) — enforced in the database via RLS

- **Operator** — the TC; runs the deal; full access.
- **Collaborator** *(built later)* — agent (own deals) or broker/compliance (oversight); no operating controls.
- **Receiving-end** — inspectors (general, termite/pest, roof, sewer), contractor, appraiser, insurance, home-warranty; gets reminders and can mark their own task done; sees nothing else.
- **Email participant** — buyer/seller, lender/title; plain email, no login.

MVP is email-first with no outside logins. Escrow full visibility is a V1 login (the first outside portal), not MVP. Tiers must be enforced by Supabase **row-level security**, not just the UI.

---

9. The vertical slice (§11) — the MVP path

One thin, complete path, email-first: a signed CA purchase agreement arrives at the dedicated deal address -> agent detects it -> **TC confirms "new deal"** -> AI extracts fields -> TC confirms -> timeline generated -> AI drafts the first lender follow-up -> TC approves and sends. Manual upload is the fallback entry at step 1.

Steps: **1 Detect -> 2 Create (HITL) -> 3 Extract -> 4 Confirm -> 5 Timeline -> 6 Draft -> 7 Send.** Each step has defined error states (wrong doc type, unreadable, missing pages, unsigned, ambiguous deal match, missing contact, send failure).

Done when: a synthetic email carrying a signed CA contract hits the dedicated deal address; the TC confirms the new deal (nothing commits without it); real fields extract with confidence; the TC confirms them; a timeline from human-verified CA rules appears; a lender follow-up is drafted; and a real email sends only after human approval — with every step in the audit log and no auto-send path anywhere.

---

10. Spec content — reconciled against everything provided

Provided across the roadmap and the spec extracts: the five rules, §8, §10, §11 (the full slice, whose "error states" column supplies the §4 "never silently guess" handling), §12, §13, and the §6 compliance **case list** (enumerated in Build Prompt 5). Only one content item is genuinely missing, plus one that is external by design:

- **§5 — the explicit list of fields to extract** from the purchase agreement, and **which are deadline-driving.** `[OBTAIN/DEFINE][BLOCKER: Phase 4]` Referenced everywhere ("pull the §5 fields") but never enumerated. If it exists in the full spec, paste it; if not, it is a decision to make, not a document to find.
- **§6 per-case thresholds** (e.g. "loan contingency approaching" = N days out) and all deadline date math. `[EXTERNAL]` These are the California deadline rules — by design they come from `ca-rules-researcher` + human verification, not the spec. See §11 (Domain knowledge and legal).

§5 — Candidate extraction field list (DRAFT — for human verification, not authoritative)

Derived from the standard California Residential Purchase Agreement (C.A.R. Form RPA) structure. **Verify against the actual form and your spec before encoding** — this is a proposal, not a source of truth. Each field is extracted with a confidence score; **deadline-driving fields must be TC-confirmed before the timeline builds** (§11 step 4). Per **Rule 2**, wiring/payment-routing details are **never** extracted; per **Rule 1**, extract only from the signed copy.

Field	Group	Deadline-driving	Notes
Buyer name(s)	Parties	No	
Seller name(s)	Parties	No	
Property address	Property	No	
Assessor's Parcel Number (APN)	Property	No	
Purchase price	Financial	No	
Initial earnest-money deposit (amount)	Financial	No	Amount only; never wiring/accoun info (Rule 2)
Increased deposit (amount, if any)	Financial	No	
Loan amount / financing type	Financial	No	
Down payment	Financial	No	

All-cash?	Financial	No	Affects which contingencies apply
Acceptance date	Dates	**Yes**	Primary trigger date for most computed deadlines
Close of escrow (COE) date	Dates	**Yes**	Often "N days after acceptance"
Earnest-money deposit due (days after acceptance)	Contingency	**Yes**	
Inspection / investigation contingency period (days)	Contingency	**Yes**	
Loan / financing contingency period (days)	Contingency	**Yes**	Or waived
Appraisal contingency period (days)	Contingency	**Yes**	Or waived
Seller-disclosure delivery period (days)	Contingency	**Yes**	
Contingency removal date(s)	Contingency	**Yes**	
Possession date / time	Dates	**Yes**	Secondary
Verification of funds deadline	Contingency	Maybe	
Loan contingency present?	Flags	No	Drives whether the loan deadline exists
Appraisal contingency present?	Flags	No	
Inspection contingency present?	Flags	No	
Buyer's agent / brokerage	Contacts	No	
Listing agent / brokerage	Contacts	No	

Escrow holder / company	Contacts	No	"Missing escrow contact" is a §6 flag
Title company	Contacts	No	
Lender / loan officer	Contacts	No	Target of the Phase 6 lender follow-up

Notes. Deadline-driving fields gate the timeline — none may create a Task until TC-confirmed (§11 step 4; Prompt 4). The exact default contingency *periods* (e.g. the standard number of days) are CA-rule content: verify via `ca-rules-researcher`, do not assume them here.

---

11. Domain knowledge and legal (EXTERNAL — no document substitutes)

These cannot be written by an AI or copied from the roadmap. They require a domain expert or lawyer and have long lead times — start them now, in parallel.

- **California deadline and contingency-removal rules.** `[EXTERNAL][BLOCKER: Phase 5]` The domain core. Trigger dates, intervals, calendar vs. business days, roll conventions, active-removal rules. Researched by `ca-rules-researcher` **with sources**, then **human-verified** before encoding. Wrong date math silently corrupts every downstream task.
- **C.A.R. form intellectual-property legal opinion.** `[EXTERNAL][BLOCKER: launch]` Confirms the Rule 1 boundary: reading a signed copy is fine; reproducing blank forms or the contingency-removal document is not. Get the written opinion before launch, not after.
- **NPI custody obligations.** `[EXTERNAL]` You will hold SSNs and bank details. Scope California breach-notification law, CCPA/CPRA duties, financial-data handling (GLBA-adjacent expectations), data retention/disposal policy, and an incident-response plan. "No-retain on the model" covers only one slice of this.
- **Unlicensed-legal-advice boundary.** `[EXTERNAL]` The tool must not advise on contingency or negotiation decisions — reminders and status only.

---

12. The seven build phases (with the prompts)

Run each in **plan mode first**, TDD, then `code-reviewer` + `compliance-auditor`, then one clean commit. Dependency order below.

Phase	Builds	Depends on	Status
1 SOR core	Schema (§10) as migrations, thin create/read API, audit log, TC auth	Stage A+B	`[WIP]` payload contract only
2 Walking skeleton	Thin email->stubs->fake send, real screens	1	`[TODO]`
3 Ingestion (real)	Detection, routing, queue, HITL, manual fallback	2	`[TODO]`
4 Extraction + confirm	Real Claude extraction, confidence, confirm gate	3	`[TODO][BLOCKER: ZDR]`
5 Compliance service	Interval service, CA rules, risk flags, drafts	1, 4	`[TODO][BLOCKER: CA rules]`
6 Draft + approve/send	Lender draft, approval UI, Postmark send, audit	2, 5	`[TODO]`
7 Dashboard + permissions	Drill-down, alerts, comm center, RLS tiers	4, 5, 6	`[TODO]`

The seven prompts (paste one per phase, in plan mode):

Prompt 1 — System of Record. Build the §10 schema as Supabase migrations; a thin FastAPI layer (create transaction, write validated Payload, read deal state); an append-only audit log written on every state change; Supabase auth for the TC (MFA on). No component reaches the DB except through the API; payloads are the only write path. Synthetic data only. Done when you can create a transaction and read its full state through the API, the audit log records each change, and tests cover schema + the payload write path. Do not build ingestion, extraction, timeline, email, or UI.

Prompt 2 — Email-triggered walking skeleton. Prove the whole email-first path with stubs and real screens: Postmark inbound webhook receiving a synthetic email + PDF; a stub ingestion step that asks the TC (HITL) "new deal or which existing?"; on confirm, write a stub payload creating the transaction; a stub extractor returning fixed §5 fields with fake confidence; extraction-review screen -> TC confirms; a trivial hardcoded timeline; an AI panel showing a stub lender draft; an "Approve & Send" button that logs a fake send. React + TS front calling the SOR API. Dedicated inbox only. Synthetic only. HITL required before any create. Done when a synthetic email walks all screens and a fake "sent" lands in the audit log. Do not use the real Claude API, real routing, or real CA rules yet.

Prompt 3 — Ingestion agent (real). Turn the stub into the real monitoring agent, decoupled (emits payloads only). Watch the dedicated address; on a signed purchase agreement propose a new transaction ID; on other docs route by ID/sender/property address or ASK the TC. HITL for both creation and updates. A lightweight queue between "detected" and "payload written." Manual-upload fallback for unreadable emails/scans. Emits only validated payloads. Dedicated address only. Synthetic

testing. Done when a synthetic PA creates a deal and a synthetic proof-of-funds attaches to the correct deal — both after TC confirmation — with routing tested and no compliance service involved.

Prompt 4 — Real extraction + confirmation gate. Send the ingested PDF to Claude (no-retain config) and extract the §5 fields, each with a confidence score, as structured JSON. "Never silently guess" handling for missing pages / unsigned / unreadable scan / wrong doc type (see §4). The confirmation gate: low-confidence and deadline-driving fields highlighted; timeline cannot build until every deadline-driving field is TC-confirmed. Manual field-entry fallback for unreadable scans. Synthetic PDFs only. Confirm no-retain is active before the first real call. Done when real fields extract with scores, a bad scan routes everything to manual, and you are BLOCKED (with explanation) if you try to proceed with an unconfirmed contingency date. Do not let any unconfirmed deadline field create a task.

Prompt 5 — Timeline compliance service. First: have `ca-rules-researcher` produce the CA deadline + active-removal rules WITH sources; human-verify before encoding. A scheduled service that reads confirmed master state and computes dated tasks with dependencies; risk flags + drafted reminders for the §6 cases (inspection not scheduled, appraisal not ordered, loan contingency approaching, earnest money not confirmed, disclosures unsigned, missing escrow contact, closing near with open tasks). Every drafted message is written back as a draft requiring human approval. Testable against synthetic master-state with no email/extraction. Deterministic date math. Never generates the Contingency Removal form (Rule 1). Done when, given a synthetic confirmed deal, the right tasks/flags/drafts appear, dates match the verified rules, and the service runs green against synthetic state alone.

Prompt 6 — Draft, approve & send, audit. Claude drafts a specific lender status request from deal state, showing WHY. Approval UI: TC edits then Approve & Send. Send via Postmark; log approver + timestamp; set a follow-up reminder if no reply. NO code path may send without a human approval (Rule 3) — the compliance-auditor must confirm. If no lender contact, ask for it. Money/wiring never touched (Rule 2). Done when a real lender email sends only after approval, the audit log shows approver + timestamp, and a codebase search finds no auto-send path.

Prompt 7 — Dashboard drill-down + permissions. Dashboard lists parties; clicking a party shows outstanding items (loan officer checklist; "2 of 3 buyers submitted proof of income"; inspection party scheduled/report received). Risk-alerts screen (prioritized) and communication center (sent/pending/replies). Enforce the four tiers via Supabase RLS so a receiving-end inspector can mark exactly one task done and see nothing else. Permissions in the database (RLS), not only UI. Every AI action shows why; every outbound needs a human tap. Done when drill-downs work and a receiving-end token can update exactly one task and read nothing else — verified by a permission test.

---

13. Supporting technical components

- **PDF pre-check library** (e.g. pypdf / pdfplumber) — page count, readability, basic checks before extraction. `[TODO]` Phase 4.

- **Scheduler** for the compliance service interval (pg_cron / APScheduler / cron is enough for MVP). `[TODO]` Phase 5.
- **Lightweight ingestion queue** — between "document detected" and "payload written" so ingestion doesn't block. No Kafka/microservices. `[TODO]` Phase 3.
- **Extraction pipeline** — PDF -> Claude -> structured JSON with per-field confidence; prompt design and confidence calibration. `[TODO]` Phase 4.
- **Routing heuristics** — match a document to a transaction by ID / sender / property address, with "ask the TC" as the safety valve. `[TODO]` Phase 3.
- **Deterministic date-math engine** for CA deadlines. `[TODO]` Phase 5.

---

14. Testing and QA

- **TDD throughout** — tests first, from each phase's "done when." Each part testable in isolation against synthetic payloads. If a part can only be tested by running the whole system, the seam is wrong.
- **Synthetic fixtures you must author** (no real NPI, ever): a signed CA purchase agreement; proof of funds; disclosures; the four inspection reports; and the failure cases — unreadable scan, unsigned doc, missing pages, wrong doc type, ambiguous deal match, multi-buyer partial submission.
- **Subagent review loop** — `code-reviewer` (fresh context) + `qa` + `compliance- auditor` on every phase; parent applies fixes.
- **Permission tests** — prove RLS enforcement (a receiving-end token can touch exactly one task).
- **End-to-end** — optional Playwright run of the full slice.

---

15. Security, privacy, and compliance posture

- **Anthropic zero-data-retention** confirmed active before any real NPI call. `[WIP][BLOCKER: real data]`
- **Row-level security policies** for the four tiers. `[TODO]` Phase 7.
- **Encryption at rest** for stored documents (Supabase storage) + TLS in transit.
- **Secrets management** — gitignored `.env`, host secret store in production; service-role and Anthropic keys backend-only; least-privilege, revocable tokens.
- **Logging discipline** — no document content, extracted field, SSN, or bank detail in logs, error messages, or an AI's context.
- **Audit log** — append-only, covering every state change, approval, and send.
- **MFA** on Supabase, Anthropic, Postmark, and GitHub admin logins. `[DONE]`
- **Data retention / disposal** policy and **incident-response** plan. `[EXTERNAL]`
- **Backups / disaster recovery** for the SOR.

---

16. Deployment and operations (Target B — production)

- **Backend hosting** for FastAPI (e.g. Fly.io / Render / Railway / Modal). `[TODO]`
- **Frontend hosting** for the React app (e.g. Vercel / Netlify). `[TODO]`
- **Domain + DNS** — a registered domain; **MX record** for a branded inbound deal address; **DKIM / SPF / DMARC** for outbound deliverability. `[TODO]`
- **Postmark production approval** + a dedicated sending domain (account is currently pending-approval; sends only to verified addresses until approved). `[TODO]`
- **Environment separation** — staging vs production; separate keys and Supabase projects.
- **CI/CD** — GitHub Actions running tests on every push; migration workflow.
- **Monitoring / error tracking** (e.g. Sentry) + uptime + alerting. `[TODO]`
- **Migrations workflow** — versioned Supabase migrations, applied per environment.

---

17. Deferred scope (Target C — explicitly NOT in the MVP)

Personal-mailbox scanning (beyond the dedicated address); Escrow V1 login (first outside portal); other party portals; CRM / MLS import; broker compliance dashboard; calendar and e-signature integrations; multi-state; daily TC briefing; cross-document inconsistency flags; agent collaborator access; more draft types. Keep manual upload as a permanent fallback. No agent teams / git worktrees / multi-agent orchestration for building — solo and sequential is right.

---

18. Definition of done and risk register

MVP done (Target A): the full §11 slice runs on synthetic data through real screens, triggered by email, with every step in the audit log and no auto-send path.

Risk register:

Risk	Mitigation
Enlarged NPI surface from email ingestion	Dedicated address only; synthetic testing; ZDR confirmed before real calls; never log NPI
False-positive transaction creation	Mandatory HITL on creation
Routing errors ("which deal?")	Agent asks the TC when unsure; never guesses
Wrong CA deadline rules	Human-verify the researcher's output; deterministic date math
Auto-send path sneaking in	compliance-auditor checks every phase
C.A.R. IP boundary	Written legal opinion before launch

Components getting coupled	If a part can't be tested alone, stop and fix the seam

---

19. MASTER CHECKLIST — every single thing

Accounts and credentials
- [x] Supabase project (MFA, US region)
- [x] Anthropic org (MFA, billing, API key)
- [x] Postmark (server, inbound address, verified sender)
- [x] GitHub private repo + PAT
- [ ] Anthropic zero-data-retention confirmed active `[BLOCKER: real data]`
- [ ] Domain registered `[for production]`
- [ ] Backend host account
- [ ] Frontend host account
- [ ] Error-monitoring account

Environment
- [x] Node 18+
- [x] Python 3.10+
- [x] git configured
- [x] Claude Code CLI

Claude Code workspace
- [x] CLAUDE.md
- [x] rules/ (security, architecture, testing, data-model, code-style)
- [x] Four subagents
- [x] Supabase MCP (read-only)
- [ ] Chrome DevTools MCP `[Phases 2, 7]`
- [ ] Optional: GitHub MCP, Playwright MCP

Spec content
- [x] Five hard rules
- [x] §8 permission tiers
- [x] §10 data model
- [x] §11 vertical slice (incl. §4 never-guess error handling)
- [x] §12 MVP scope, §13 priority
- [x] §6 compliance case list (from Build Prompt 5)
- [~] §5 extraction field list — candidate drafted (§10); pending human verification `[BLOCKER: Phase 4 until verified]`
- [ ] §6 per-case thresholds = CA deadline rules `[EXTERNAL]` (see domain/legal)

Domain and legal (external)
- [ ] CA deadline + contingency-removal rules, researched + human-verified `[BLOCKER: Phase 5]`
- [ ] C.A.R. form IP legal opinion `[BLOCKER: launch]`
- [ ] NPI custody obligations (CA breach law, CCPA/CPRA, retention/disposal, incident response)
- [ ] Unlicensed-legal-advice boundary confirmed

Build phases
- [~] Phase 1 — SOR core (payload contract + skeleton done; schema/migrations/API/audit/auth remaining)
- [ ] Phase 2 — walking skeleton

- [ ] Phase 3 — ingestion (real)
- [ ] Phase 4 — extraction + confirm `[BLOCKER: ZDR, §5]`
- [ ] Phase 5 — compliance service `[BLOCKER: CA rules, §6]`
- [ ] Phase 6 — draft, approve & send
- [ ] Phase 7 — dashboard + RLS permissions

Supporting components
- [ ] PDF pre-check library
- [ ] Scheduler
- [ ] Ingestion queue
- [ ] Extraction pipeline + confidence calibration
- [ ] Routing heuristics
- [ ] Deterministic date-math engine

Testing
- [x] Payload contract tests
- [ ] Synthetic fixtures (PA, proof of funds, disclosures, inspection reports, failure cases)
- [ ] Per-phase code-reviewer + qa + compliance-auditor passes
- [ ] Permission (RLS) tests
- [ ] Optional end-to-end (Playwright)

Security and compliance
- [x] MFA on all admin logins
- [x] Secrets in gitignored .env / password manager
- [ ] RLS policies for four tiers
- [ ] Encryption at rest for documents
- [ ] Logging discipline enforced (no NPI)
- [ ] Audit log on every state change
- [ ] Data retention/disposal + incident-response plan

Deployment and operations (production)
- [ ] Backend deployed
- [ ] Frontend deployed
- [ ] Domain + MX + DKIM/SPF/DMARC
- [ ] Postmark production approval + sending domain
- [ ] Staging vs production separation
- [ ] CI/CD (tests on push, migrations)
- [ ] Monitoring / alerting / error tracking
- [ ] Backups / disaster recovery

---

20. Current status snapshot (2026-07-12)

Done: Stage A (all four services live and proven; keys in gitignored `.env`).
Stage B (CLAUDE.md, rules/, four subagents, Supabase MCP read-only, Node installed).
A Phase 1 down-payment: the `Payload` contract + backend skeleton + 12 passing tests.
Four commits on `main`.

Immediate next: finish Phase 1 properly (schema + migrations + transaction API + audit log + TC auth) in plan mode, in the tc-mvp Claude Code session, then Phase 2.

Parallel, long-lead items to start now: Anthropic ZDR approval; verify the drafted §5 extraction field list (§10); commission the CA rules research + human verification; get the C.A.R. IP legal opinion and a read on NPI custody obligations.

End of document.